

**NAME:** M.Greeshma

**COURSE:** Design and Analysis of Algorithms

**CODE:** CSA0619

## **REPORT**

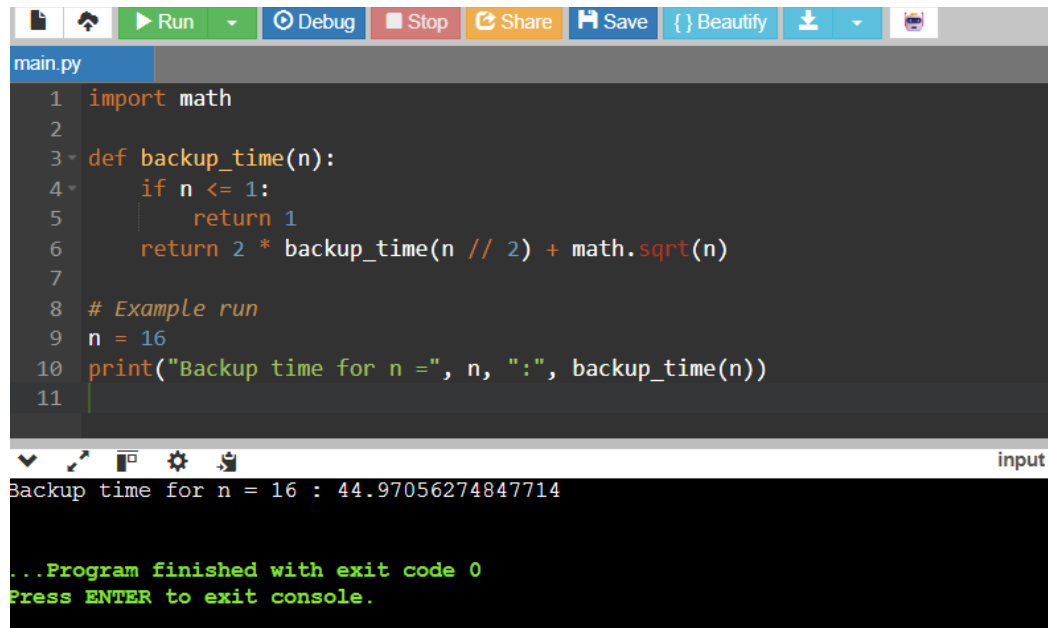
### **Q9. Application: Data Backup Optimization System**

A backup system uses an algorithm with recurrence  $T(n) = 2T(n/2) + \sqrt{n}$ . Apply the Master Theorem to determine the time complexity. Compare  $f(n)$  with  $n^{\log_b a}$  and identify the correct case. Analyze performance for large datasets. Interpret efficiency.

#### **1. Source Code (Python Example)**

```
import math
def backup_time(n):
    if n <= 1:
        return 1
    return 2 * backup_time(n // 2) + math.sqrt(n)
# Example run
n = 16
print("Backup time for n =", n, ":", backup_time(n))
```

## 2.OUTPUT



```
main.py
1 import math
2
3 def backup_time(n):
4     if n <= 1:
5         return 1
6     return 2 * backup_time(n // 2) + math.sqrt(n)
7
8 # Example run
9 n = 16
10 print("Backup time for n =", n, ":", backup_time(n))
11
```

Backup time for n = 16 : 44.97056274847714

...Program finished with exit code 0  
Press ENTER to exit console.

We are given the recurrence:

$$T(n) = 2T\left(\frac{n}{2}\right) + \sqrt{n}$$

- Parameters:
  - $a = 2$ (number of subproblems)
  - $b = 2$ (factor by which problem size is reduced)
  - $f(n) = \sqrt{n}$

Now compute:

$$n^{\log_b a} = n^{\log_2 2} = n^1 = n$$

So we compare  $f(n) = \sqrt{n}$  with  $n$ .

- Case 1: If  $f(n) = O(n^c)$  where  $c < \log_b a$ , then  $T(n) = \Theta(n^{\log_b a})$ .
- Case 2: If  $f(n) = \Theta(n^{\log_b a})$ , then  $T(n) = \Theta(n^{\log_b a} \log n)$ .
- Case 3: If  $f(n) = \Omega(n^c)$  where  $c > \log_b a$ , then  $T(n) = \Theta(f(n))$ .

Here:

- $f(n) = \sqrt{n} = n^{1/2}$

- $\log_b a = 1$   
So  $f(n) = n^{1/2}$ , which is **polynomially smaller** than  $n^1$ .  
☞ This matches **Case 1**.

$$T(n) = \Theta(n)$$

### ANALYSIS:

- For **large datasets**, the algorithm runs in **linear time**.
- The additional cost of  $\sqrt{n}$  per level is negligible compared to the dominant  $n$ .
- This means the backup system scales efficiently, even with millions of files.

### Q25. Application: Fraud Detection System

A fraud detection algorithm follows the recurrence  $T(n) = 2T(n/2) + n^2 \log n$ . Apply the Master Theorem to determine the time complexity. Compare  $f(n)$  with  $n \log b$  and justify the correct case. Analyze performance for large transaction datasets. Interpret efficiency clearly.

### 1. Source Code (Python):

```
import math
def fraud_detection_time(n):
    # Base case
    if n <= 1:
```

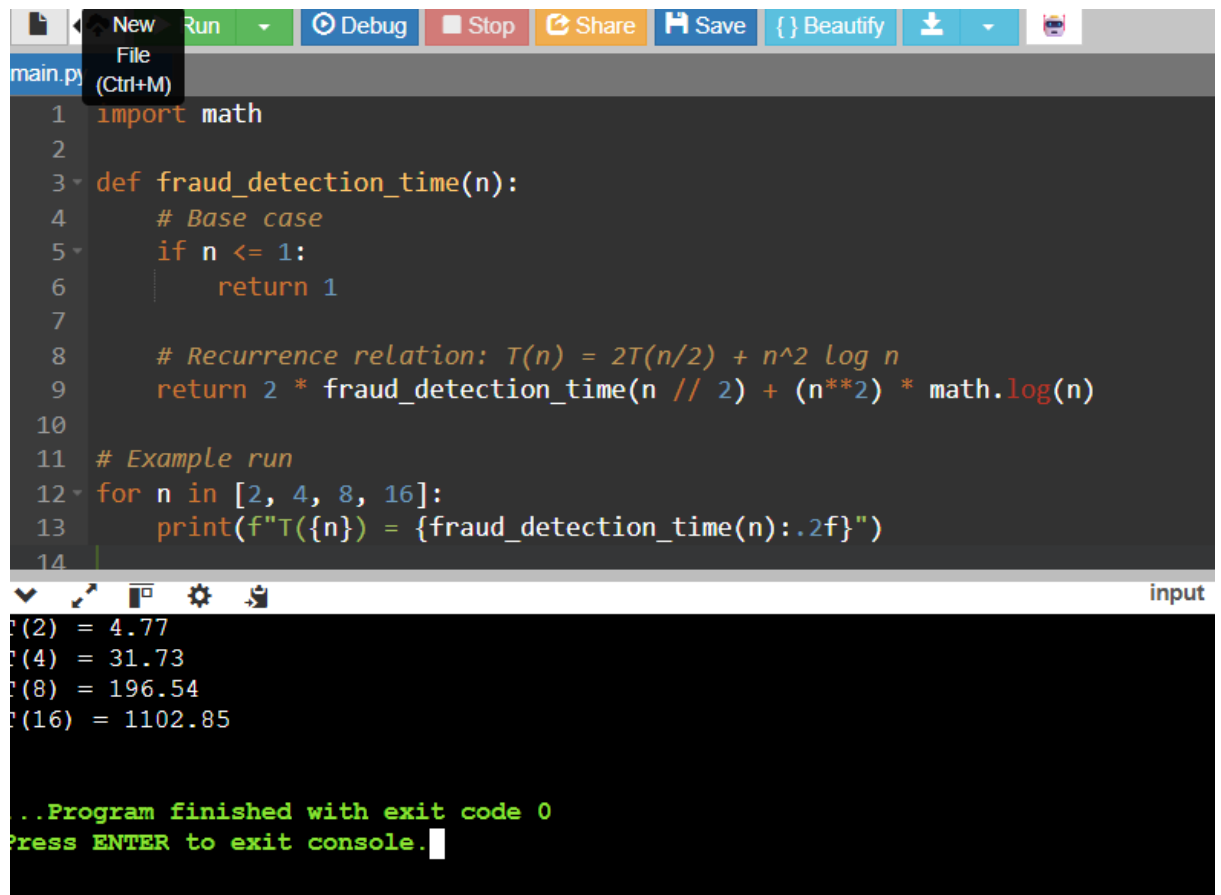
```
return 1
```

```
# Recurrence relation:  $T(n) = 2T(n/2) + n^2 \log n$   
return 2 * fraud_detection_time(n // 2) + (n**2) *  
math.log(n)
```

```
# Example run
```

```
for n in [2, 4, 8, 16]:  
    print(f"T({n}) = {fraud_detection_time(n):.2f}")
```

## 2. OUTPUT:



The screenshot shows a code editor with a dark theme. The top toolbar includes buttons for 'New', 'Run', 'Debug', 'Stop', 'Share', 'Save', 'Beautify', and a download icon. The code is written in Python and defines a function `fraud_detection_time(n)` that implements the recurrence relation  $T(n) = 2T(n/2) + n^2 \log n$ . The base case is  $T(1) = 1$ . An example run is provided for  $n \in [2, 4, 8, 16]$ . The output is displayed in a console window at the bottom, showing the results of the function calls.

```
main.py  
1 import math  
2  
3 def fraud_detection_time(n):  
4     # Base case  
5     if n <= 1:  
6         return 1  
7  
8     # Recurrence relation:  $T(n) = 2T(n/2) + n^2 \log n$   
9     return 2 * fraud_detection_time(n // 2) + (n**2) * math.log(n)  
10  
11 # Example run  
12 for n in [2, 4, 8, 16]:  
13     print(f"T({n}) = {fraud_detection_time(n):.2f}")  
14
```

input

```
T(2) = 4.77  
T(4) = 31.73  
T(8) = 196.54  
T(16) = 1102.85  
  
..Program finished with exit code 0  
Press ENTER to exit console.
```

We are given:

$$T(n) = 2T\left(\frac{n}{2}\right) + n^2 \log n$$

- $a = 2$ (number of subproblems)
- $b = 2$ (factor by which problem size is reduced)
- $f(n) = n^2 \log n$

**Compare with  $n^{\log_b a}$**

$$n^{\log_b a} = n^{\log_2 2} = n^1 = n$$

So we compare  $f(n) = n^2 \log n$  with  $n$ .

Clearly:

$$n^2 \log n \gg n$$

That means  $f(n)$  grows **polynomially faster** than  $n^{\log_b a}$ .

- **Case 1:**  $f(n)$  smaller  $\rightarrow$  not applicable.
- **Case 2:**  $f(n)$  same order  $\rightarrow$  not applicable.
- **Case 3:**  $f(n)$  larger  $\rightarrow$  applicable.

☞ This matches **Case 3**.

$$T(n) = \Theta(n^2 \log n)$$

**ANALYSIS:**

- For **large transaction datasets**, the algorithm runs in **super-quadratic time**.
- The  $n^2 \log n$  term dominates, meaning runtime grows faster than quadratic but slower than cubic.
- As dataset size doubles, runtime increases more than fourfold due to the logarithmic multiplier.